

Employee Management System

Eureka Service Discovery — Implementation & Interview Guide

Purpose: document what we implemented, why Eureka was needed, how the runtime flow works, and how to explain the design in a Java/Spring Boot microservices interview.

Current milestone

- Eureka Server is running as the service registry on port 8761.
- EMS Auth, EMS Backend, and EMS Notification Service register with Eureka.
- EMS Backend discovers Notification Service by logical service ID.
- Spring Cloud LoadBalancer resolves the service ID to an available instance.
- Spring Retry remains responsible for configured retryable notification failures.
- API Gateway still uses static localhost routes; Eureka-based Gateway routing is the next step.

Prepared for interview practice • October 2026

1. Executive Summary

The EMS application started with fixed URLs between services. That is manageable when each service has one known instance, but it tightly couples the caller to the callee's physical host and port. In a microservices ecosystem, services may be restarted, moved, or horizontally scaled.

Eureka Service Discovery introduces a service registry. Service instances register themselves with Eureka, and consumers discover a service by logical service ID instead of knowing the exact physical location.

In the current implementation, the key flow is EMS Backend → Notification Service. The backend uses EMS-NOTIFICATION-SERVICE as the logical identity. Spring Cloud LoadBalancer resolves that identity to an available instance.

Interview answer: “We introduced Eureka to remove hard-coded service-to-service locations. Services register themselves with a central registry, consumers discover them by logical service name, and Spring Cloud LoadBalancer selects an available instance. This makes horizontal scaling and endpoint changes easier to handle.”

2. Current EMS Ecosystem

Component	Role	Port
React Frontend	Browser UI; calls API Gateway	3000
API Gateway	Browser-facing entry point; static routes for now	8082
EMS Backend	Employee/Department/Office APIs; discovers Notification Service	8080
Auth Service	Registration/login; JWT issuance	8083
Notification Service	Notification API + persistence	8081
Eureka Server	Service registry and discovery metadata	8761

The separate block-diagram PDF shows the current architecture, runtime notification flow, and the before/after service-discovery model.

3. Why Did We Need Eureka?

Core problem: location coupling. A URL such as `http://localhost:8081/...` contains both the identity of the service and the physical location of one instance.

- **Scaling:** multiple Notification Service instances cannot be represented by one fixed URL.
- **Deployment:** a service may move to another host, VM, container, or port.
- **Configuration:** endpoint changes otherwise propagate into consumer configuration.
- **Availability:** a fixed address may point to an instance that is unavailable.
- **Microservices principle:** consumers should depend on a stable logical service identity rather than an individual machine location.

Eureka solves service **registration and discovery**. It does not itself perform the business HTTP call, retry failed requests, authenticate users, or provide circuit breaking.

The resulting separation is: **Eureka = where?** → **LoadBalancer = which instance?** → **RestClient = make the HTTP call** → **Spring Retry = retry a configured failure**.

4. What We Implemented

4.1 Eureka Server

```
spring.application.name=ems-service-registry
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

- Runs on port 8761 and acts as the service registry.
- It does not register with another Eureka server.

4.2 Registered applications

- **EMS-AUTH-SERVICE** — authentication service, port 8083.
- **EMS-BACKEND** — main business service, port 8080.
- **EMS-NOTIFICATION-SERVICE** — notification service, port 8081.

4.3 Eureka client settings

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
```

The consumer needs to know where the registry is; it does not need a hard-coded URL for every service it may call.

4.4 Backend → Notification Service

The notification client now uses the logical service ID rather than the physical localhost endpoint:

```
http://EMS-NOTIFICATION-SERVICE/notification-service/api/notifications
```

The EMS Backend uses a load-balanced `RestClient.Builder`. The service ID is resolved to a concrete Notification Service instance at runtime.

4.5 Retry was retained

```
@Retryable(
    retryFor = ResourceAccessException.class,
    maxAttemptsExpression = "${notification.retry.max-attempts:3}",
    backoff = @Backoff(
        delayExpression = "${notification.retry.backoff-delay:1000}"
    )
)
```

Discovery and retry are deliberately separate responsibilities.

5. End-to-End Runtime Flow

- 1. React sends the employee request through the API Gateway.
- 2. The Gateway currently routes the request to EMS Backend using its existing static route.
- 3. EMS Backend performs the employee database transaction.
- 4. After commit, `@TransactionalEventListener(AFTER_COMMIT)` processes the employee event.
- 5. The notification client creates the notification request.
- 6. The client requests `EMS-NOTIFICATION-SERVICE` by logical name.
- 7. Discovery + Spring Cloud LoadBalancer resolve an available instance.
- 8. `RestClient` sends the notification HTTP request.
- 9. Spring Retry retries the configured retryable failure condition when necessary.
- 10. Notification Service returns the HTTP response.

Latest successful test: after a clean Maven build, employee ID 29 was committed, the `AFTER_COMMIT` event ran, the notification client attempted the logical Notification Service endpoint, and the notification succeeded on attempt 2 with HTTP 201 CREATED. The employee data remained committed and was subsequently retrievable.

This demonstrates the runtime path rather than merely showing configuration.

6. What Happens When We Scale?

```
EMS-NOTIFICATION-SERVICE
■■■ instance A → host/port A
■■■ instance B → host/port B
■■■ instance C → host/port C
```

All instances register under the same logical service ID. The consumer continues to call EMS-NOTIFICATION-SERVICE; it does not need three physical URLs. The LoadBalancer obtains the available instances and selects one.

Strong interview line: “The service ID is the stable identity; the instance address is ephemeral infrastructure data.”

Benefits

- Horizontal scaling without changing consumer code.
- Dynamic registration of instances.
- Reduced endpoint/configuration coupling.
- Better support for independently deployed microservices.

Trade-offs

- The registry becomes an infrastructure dependency and must be operated reliably.
- Discovery adds operational and conceptual complexity compared with a single fixed URL.

7. Why the API Gateway Is Not Eureka-Based Yet

We intentionally stopped the current milestone at backend-side discovery. This isolates the learning objective: first prove that Eureka can register services and that one consumer can discover another service.

The Gateway still has routes whose URI points to localhost, for example:

```
spring.cloud.gateway.server.webflux.routes[2].uri=http://localhost:8080
```

The next milestone is to make the API Gateway an Eureka client and use discovery-based routes conceptually like:

```
uri=lb://EMS-BACKEND
```

Then the browser-facing Gateway will also depend on logical service identity rather than a fixed backend host/port. The learning sequence is: **Eureka Server** → **register services** → **backend discovers Notification Service** → **validate scaling/load balancing** → **Gateway becomes Eureka client** → **Gateway uses lb:// routes**.

8. Interview Questions & Model Answers

Q. What is service discovery?

Answer: A mechanism for finding a service's runtime network location using a logical service identity instead of a statically configured host and port.

Q. Why did you introduce Eureka?

Answer: To remove hard-coded service-to-service location coupling and support dynamic discovery when services are restarted, moved, or horizontally scaled.

Q. What is Eureka Server?

Answer: A service registry that maintains information about registered service instances and makes that information discoverable to clients.

Q. Is Eureka a load balancer?

Answer: No. Eureka supplies discovery information. Spring Cloud LoadBalancer uses the discovered instances to select an instance for a request.

Q. Does Eureka make the HTTP call?

Answer: No. RestClient makes the HTTP call after the service name has been resolved.

Q. What happens with three Notification instances?

Answer: All three register under the same service ID. The consumer discovers the instances and LoadBalancer selects one.

Q. Did Eureka replace Spring Retry?

Answer: No. Eureka/LoadBalancer locates and selects an instance; Spring Retry handles configured transient failures.

Q. Why AFTER_COMMIT?

Answer: So the employee transaction commits independently. Notification failure should not roll back employee persistence.

Q. Is the notification call asynchronous?

Answer: The event is triggered after commit, but the notification HTTP call itself is synchronous.

Q. What is your next step?

Answer: Make the API Gateway an Eureka client and replace fixed HTTP routes with lb:// service routes.

9. Failure-Driven Learning Notes

During the discovery implementation, introducing a load-balanced RestClient builder initially interacted with the Eureka client's own HTTP infrastructure. Logs showed LoadBalancer activity trying to resolve `localhost` as a service ID and a bean-creation cycle around discovery components.

The engineering lesson is important: **cross-cutting client configuration must be scoped carefully**. A load-balanced client should not accidentally become the transport used by framework infrastructure that itself talks to Eureka.

A later employee-creation failure showed `NoClassDefFoundError: NotificationClientImpl$1` followed by a misleading recovery-method message. The synthetic `$1` class indicated a stale/incremental compilation artifact. A full Maven clean build resolved the issue.

- Read the deepest/root cause rather than only the final framework exception.
- Do not change working retry/recovery logic to fix a compiler-generated class-loading artifact.
- After structural dependency/configuration changes, a clean build is a useful diagnostic step.

10. One-Minute Interview Cheat Sheet

Problem: “Our microservices used fixed host/port URLs, creating location coupling.”

Solution: “We introduced Eureka as a service registry. Services register themselves and consumers discover them by logical service ID.”

Implementation: “EMS Backend uses a load-balanced RestClient to call EMS-NOTIFICATION-SERVICE by logical name.”

Load balancing: “Spring Cloud LoadBalancer selects an available instance from the discovered instances.”

Resilience: “Spring Retry handles configured retryable failures; discovery and retry are separate concerns.”

Transaction: “Notification is triggered after the employee transaction commits using AFTER_COMMIT.”

Current limitation: “The API Gateway still uses static routes. The next step is to make it an Eureka client and use lb:// routes.”

Architecture principle: “Stable logical service identity, dynamic physical instance location.”

If asked why not hard-code the URL: “Hard-coding is acceptable for a simple single-instance local system, but in a microservices ecosystem instances can scale or move. Service discovery lets the consumer depend on service identity while infrastructure determines the current instance location.”

Scope note: this document describes the EMS state immediately before converting the API Gateway to Eureka-based routing. It does not claim that Gateway discovery has already been implemented.